

Test case prioritization—ANT algorithm with faults severity

ANDREEA VESCAN*, *Computer Science Department, Babes-Bolyai University, 400084 Cluj-Napoca, Romania.*

CAMELIA-M. PINTEA**, *Department of Mathematics and Informatics, Technical University of Cluj-Napoca, 430122 Baia-Mare, Romania.*

PETRICĂ C. POP†, *Department of Mathematics and Informatics, Technical University of Cluj-Napoca, 430122 Baia-Mare, Romania.*

Abstract

Regression testing is applied whenever a code changes, ensuring that the modifications fixed the fault and no other faults are introduced. Due to a large number of test cases to be run, test case prioritization is one of the strategies that allows to run the test cases with the highest fault rate first. The aim of the paper is to present an optimized test case prioritization method inspired by ant colony optimization, *test case prioritization—ANT*. The criteria used by the optimization algorithm are the number of faults not covered yet by the selected test cases and the sum of severity of the faults. The cost, i.e. time execution, for test cases is considered in the computation of the pheromone deposited on the graph's edges. The *average percentage of fault detected* metric, as best selection criterion, is used to uncover maximum faults with the highest severity, and reducing the regression testing time. Several experiments are considered, detailed and discussed, comparing various algorithm parameter's alternatives. A benchmark project is also used to validate the proposed approach. The obtained results are encouraging, being a cornerstone for new perspectives to be considered.

Keywords: Regression testing, security features, faults severity, prioritization, ant algorithms, optimization.

1 Introduction

In order to successfully ensure high quality for modified software systems, regression testing is one of the activities that should be included. Testing should be a well-conducted process, where security-related faults must be revealed as soon as possible. Test case prioritization is one strategy that can be applied to discover and run them early within the limited time block.

Faults may be characterized by different severity and different priority [3] and when prioritizing which test cases to execute next, those should be considered. Severity by definition describes the gravity of an undesirable occurrence. Thus, in testing, severity defines the extent to which a particular defect could create an impact on the application or system.

Various strategies exists for regression testing, i.e. test case selection, test case minimization and test case prioritization, each having their benefits and weaknesses. Applying regression testing ensures the stability and correctness of a program when modifications are made: first to ensure that

*E-mail: avescan@cs.ubbcluj.ro

**E-mail: dr.camelia.pintea@ieec.org

†E-mail: petrica.pop@cunbm.utcluj.ro

the modification fixed the identified bug and that no new bugs were introduced. When prioritizing test cases various criteria are used: the fault detection rate criterion [35], rate of detection of high-risk faults, regression errors and confidence of the reliability criterion [21]. Non-functional quality attributes [2] should be considered during regression testing and further investigation is needed.

Regression testing is a complex optimization problem. Nowadays, the NP-hard problems are mainly solved using machine learning tools as e.g. including human in the loop [31, 32] and heuristics [14, 33].

Bio-inspired algorithms proved to be efficient learning mechanisms for solving complex optimization problems. Ant colony optimization [15] is used here as the basis for the *test case prioritization-ANT (TCP-ANT)* that uncovers maximum faults, each fault reviewing a ‘severity’ value and reducing the regression testing time (considering cost as test case execution time). The *average percentage of fault detected (APFD)* metric [35] is used.

The aim of this paper is to investigate the test case prioritization problem, proposing an ant-based optimization algorithm with various criterion: number of covered faults, cost of execution and severity of faults, and to further validate it by considering four configuration systems (test cases and faults), and one benchmark project.

The rest of the paper is organized as follows: Section 2 presents the main definitions regarding regression testing, test case prioritization and used metrics, and the state of art. Section 3 illustrates the *TCP-ANT* technique. Section 4 describes the undertaken experiments, followed by a discussion of the achieved results. Section 5 presents the threats to validity and what measures we take to counteract them. Some further work ideas and the conclusions reiterate the *TCP-ANT* promising results and considering other security faults features.

2 Regression Testing Techniques and Test Case Prioritization

This section contains information about regression testing, details about test case prioritization and a short related work.

2.1 Regression testing techniques

Test case management [11] can be categorized into three investigation areas: test suite minimization, test case selection and test case prioritization. The developed approaches aim to reduce the cost and time during regression testing, thus a better efficiency:

- **test suite minimization** – the aim is to eliminate redundant test cases in order to reduce the number of tests to run;
- **test case selection** – the aim is to identify a subset of the test suite that is relevant to some set of recent modifications;
- **test case prioritization** – the aim is to find an optimal order of the test cases in such a way that early fault detection is maximized.

A detailed view about subtypes of regression testing is provided in [10]. Further research indicated test suite augmentation as another type of regression testing along with several subtypes and subcategories [10]. Therefore, the main types of RT are TCS, TCP, test suite reduction (also known as test suite minimization) and test suite augmentation. Figure 1 illustrates the subtypes of regression testing.

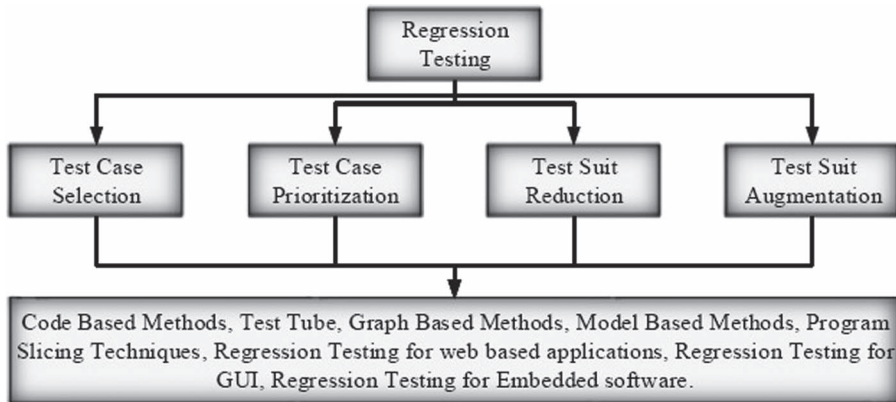


FIGURE 1. Subtypes of regression testing (RT) techniques [10].

The definition of the **regression test** [17] follows.

DEFINITION 2.1

Let P be a procedure or program; let P' be a modified version of P ; and let T be a test suite for P . A typical **regression test** proceeds as follows:

1. Select $T' \subseteq T$, a set of test cases to execute on P' .
2. Test P' with T' , establishing P' 's correctness with respect to T' .
3. If necessary, create T'' , a set of new functional or structural test cases for P' .
4. Test P' with T'' , establishing P' 's correctness with respect to T'' .
5. Create T''' , a new test suite and test execution profile for P' , from T , T' and T'' .

The steps of the previous definition [17] rise some issues. A regression testing method could retest the entire method, safely executing all test cases. The time and cost efficiency of regression testing could be reached while using: *test case selection techniques* to reduce the number of test cases (by a given criterion) and *test case prioritization techniques* to order the test cases (by the rate of early fault detection).

2.2 Test case prioritization. Definition

The **test case prioritization problem** [17, 35] is defined as follows:

DEFINITION 2.2

Given a test suite (T), the set of permutations of T (PT) and a function from PT to real numbers (f), the problem is to find $T' \in PT$ such that $(\forall T'') (T'' \in PT) (T'' \neq T) [f(T') \geq f(T'')]$.

In the previous definition, the function f illustrates the unknown rate of fault detection. The *APFD* metric [35] is used for $f : PT \rightarrow \mathbb{R}$ in order to early detecting the faults. In order to solve the *test case prioritization* problem, diverse techniques were involved. There were also considered investigations to increase the rate of fault detection, total statement coverage and function coverage [16, 23, 35].

2.3 Related work

Kavitha [20] proposed a test case prioritization algorithm that is based on severity of fault, i.e. two factors rate of fault detection and fault impact for prioritizing test cases being proposed. Two industrial projects are used to validate the approach.

An approach to automatically selecting test cases for security policy evolution [24] was proposed in [18].

The main security approaches were operationalized by Potter and McGraw [34], distinguishing testing security mechanisms to ensure that their functionality were properly implemented. Performing risk-based security testing motivated them by understanding and simulating the attacker's approach.

Zhang *et al.* [38] changed the priorities and the cost of the test case. A new metric to evaluate rate of fault dependency was proposed by Graves *et al.* [19], the new algorithm using the fast dependency among the faults.

Various heuristics are used to solve the *test case prioritization* problem: genetic algorithm [22] and ant colony optimization algorithm [12, 13, 37]. The genetic algorithm [22] had better results when compared with some greedy versions and hill climbing to prioritize test cases. The survey paper [36] includes various approaches for the test case prioritization problem.

A time constraint environment is used in the [37] approach. In the new ant-based approach, the TCP-ANT algorithm is also considered *the remaining set of uncovered faults* and obtained multiple solutions in a running process. The presented papers considered the cost, the faults and fault dependencies. The APFD metric is also used and comparisons with the unprioritized approach are made.

Furthermore, the proposed TCP-ANT algorithm is described.

3 Test Case Prioritization–ANT Algorithm

The test case prioritization algorithm model proposed at first in [8] was inspired by the Elbaum *et al.* [7] approach that incorporated test costs and faults severity into the prioritization of test cases.

The current work makes a step forward, as it includes several experiments, validating the approach with two more case studies: one theoretical and one based on real project.

We recall first the metrics used in our approach [8]. Furthermore, a description of TCP-ANT is provided.

3.1 Definitions and metrics

Regarding the optimization measures in regression testing, one of the most widely used metrics is APFD, which measures *the effectiveness of TCP as the rate of fault detection* achieved by the produced ordering of test cases [6].

APFD [6] relates with earlier fault detection abilities.

DEFINITION 3.1

Let T be a test suite containing m test cases, and F be a set of n faults detected by T . Let ftf_i be the first test case in an ordering σ of T that reveals fault i . Given an ordering σ of the test suite T , the APFD metric is defined as follows.

$$APFD(\sigma) = 1 - \frac{\sum_{i=1}^q ftf_i}{m \times n} + \frac{1}{2 \times m} \quad (3.1)$$

The APFD [16] metric assumes that all faults have the same severity (i.e. they have equal cost) and all test cases take the equal effort to execute. A higher APFD value, $APFD \in [0..100]$, implies a higher fault detection rate. In *TCP-ANT*, the APFD factor define the best solution.

Following the definitions of the *rate of remaining fault detection (RRTF)* based on the RTF and the *percentage of remaining fault detected (PRFD)* based on PFD considering the *remaining* set of faults to be detected.

DEFINITION 3.2

The RRTF [8] is the average number of defects found per minute by a test case, from the remaining set of uncovered faults.

For the test case T_j , $RRTF_j$, equation (1), use the number of defects found by T_j from the remaining set of uncovered faults and the time used by T_j to detect the same defects.

RN_j is the number of remaining faults detected by T_j test case and $time_j$ the time used by T_j test case.

$$RRTF_j = \frac{RN_j}{time_j} \times 100. \quad (3.2)$$

DEFINITION 3.3

The PRFD [8] for test case T_j is computed with the number of faults found by test case T_j from the set of remaining faults to be covered and total number of faults.

The PRFD is expressed as in Equation (3.3), where RN_j is the number of remaining faults detected by T_j test case and N the total number of faults.

$$PRFD_j = \frac{RN_j}{N} \times 100. \quad (3.3)$$

3.2 Description of the TCP-ANT algorithm

The *TCP-ANT* algorithm is based on the *ant colony system (ACS)* [15]. The factors *APFD*, *RRTF*, *PRFD* and the cost of the test cases are used. During the selection of the next test case, there are considered the uncovered faults of test case and the severity of faults. During the local update of the pheromone level, the execution cost is used.

At first, the ants are placed within the nodes, i.e. the test cases. The initialization of the parameters including the pheromone trails is done.

The main phase of the algorithm, the solution construction, includes a probabilistic selection of next tour node (test case) from available test cases from neighbourhood. The selection of next node (test case) is based on the number of faults of the test case (from uncovered yet faults) and the sum of severity of the faults that are not uncovered yet. The pheromone trails are also considered.

Remark 1) For the severity of faults we have used a linear scale from 1 through 8 as provided and justified in the approach [7].

Remark 2) These values (number of faults uncovered yet and sum of faults severity) are not usually defined in the same domain. As a result, they are normalized to values in the interval [0, 1] with the min-max normalization. The min-max normalization performs a linear transformation on the original data values. Suppose that $minX$ and $maxX$ are the minimum and maximum of feature X . We would like to map interval $[minX, maxX]$ into a new interval $[new_minX, new_maxX]$.

Consequently, every value v from the original interval will be mapped into value new_v using the following formula:

$$new_v = \frac{v - \min X}{\max X - \min X}.$$

Min-max normalization preserves the relationships among the original data values.

After an iteration, a solution (a tour of test cases, thus a prioritization) with a maximum fault detection rate at minimum cost and maximal/highest severity faults is found. As in ACS, after each step, a local update rule follows.

The last phase includes the global pheromone update rule and finding the best global solution. The algorithm stops after a given number of iterations.

A solution is assessed based on the APFD value.

The *TCP-ANT* algorithm reaches goals similar to other related methods when considering covered faults and cost. The *TCP-ANT* differs by the fact that the solution of the prioritization is verified against duplicate coverage of faults by test cases. The prioritization solution ends with test cases that do not discover new faults.

4 Experiments, Numerical Results and Discussions

The *TCP-ANT* algorithm was implemented in C/C++ and it is based on a version of the algorithm developed by Perez-Urbe [25] in order to solve the classical *traveling salesman problem (TSP)*. The experiments consider various criterion combinations (faults, costs, number of test cases, severity), considering the analysis of replication strategy [5] and the configuration [20] approach, and to assign the faults severity we used the function criticality presented in [7].

The *TCP-ANT* parameters for the tests are the following: three, four and five ants, ten iterations, $q_0 = 0.5$; a parameter to control the influence of pheromone, $\alpha = 1$; a parameter to control the influence of the desirability of state transition $\beta = 2$; the pheromone evaporation coefficient, $\rho = 0.0001$ and the initial amount of pheromone deposited on the graph's edges $\tau_0 = 0.01$.

Remark 1.) The above-stated parameters were used in our first related work [8]. In the current investigation, after an in-depth analysis, we found more suitable values for the parameters.

In order to build confidence for the ant-based technique we used one hundred runs for each test suite. Each run yielded the path found whether or not optimal.

In the first experiments [8] we considered only five test cases and five faults, with the cost associated to the test cases and severity associated to the faults.

In order to validate at a higher/complex level the previous obtained results, the current experiments include more test cases for the prioritization and a real-world application. According to [5] statistical generalization is the common way when doing surveys, but in doing case studies the analytic generalization should be used. Multiple cases resemble multiple experiments and under these circumstances the mode of generalization is analytic. If two or more cases are shown to support the same theory, replication [5] may be claimed.

Our replication strategy: for each experiment the number of ants, number of faults and number of test cases were modified, and also the severity values were varied.

The description of modified elements for each experiment follows:

- Experiment 1: 10 test cases, 10 faults and associated severity for each fault (with values: 2, 4, 6, 8).

TABLE 1. Replication strategy of the theoretical and the real-life project *grep* [9] experiments.

Characteristics	Theoretical			Real
	<i>Exp 1</i>	<i>Exp 2</i>	<i>Exp 3</i>	<i>Exp 4</i>
# Test cases	10	15	10	465
# Faults	10	10	15	4
# Severity values	{2, 4, 6, 8}	{2, 4, 6, 8}	{1, 2, 3, 4, 5, 6, 8}	{2, 3, 4, 5}

TABLE 2. Experimental instance: test cases, faults identified, cost and severity of faults based on [20].

<i>Testcase/faults</i>	<i>F1</i>	<i>F2</i>	<i>F3</i>	<i>F4</i>	<i>F5</i>	<i>F6</i>	<i>F7</i>	<i>F8</i>	<i>F9</i>	<i>F10</i>	<i>Cost</i>
T1	0	0	0	0	0	0	0	1	0	1	9
T2	0	1	0	1	0	0	0	0	0	0	8
T3	0	1	0	1	0	0	0	0	0	0	14
T4	0	0	1	0	0	0	1	0	1	0	9
T5	0	1	0	0	0	0	1	0	0	0	12
T6	0	0	1	0	0	0	0	1	1	0	14
T7	0	0	0	0	0	0	1	0	0	0	11
T8	1	0	0	0	1	1	0	0	0	1	10
T9	1	0	0	0	0	1	0	0	0	0	10
T10	0	0	1	0	0	0	0	0	1	0	13
<i>Severity</i>	8	4	2	4	6	4	4	4	4	2	

- Experiment 2: 15 test cases, 10 faults and associated severity for each fault (with values: 2, 4, 6, 8).
- Experiment 3: 10 test cases, 15 faults and associated severity for each fault (with values ranging from 1 to 8).
- Experiment 4: 465 test cases, 4 faults and associated severity for each fault (with values ranging from 2 to 5).

In order to evaluate our approach, we analysed the experiments. Table 1 summarizes the properties of the experiments: the number of test cases, the number of faults and severity values.

4.1 Experiment 1. Theoretical configuration

The experiments use the configuration from the [20] paper. Table 2 shows the data set. It consists of the test suite of test cases, the exposed faults of each test case, the cost of test cases and the severity of each fault detected by the test cases.

The APFD for the initial configuration (T1, T2, T3, T4, T5, T6, T7, T8, T9, T10) is 0.63.

This experiment aimed at discovering the best values for the number of ants to be used in the test case prioritization ant colony optimization algorithm. Thus, we have conducted three different runs, considering 3, 4 and 5 ants, with 100 runs for each experiment. The best solutions in terms of APFD were recorded for each run of the total 100 runs.

Figure 2 illustrates the mean of all the 100 runs for each different experiment with 3–5 ants. We observe that the best solution is obtained using five ants.

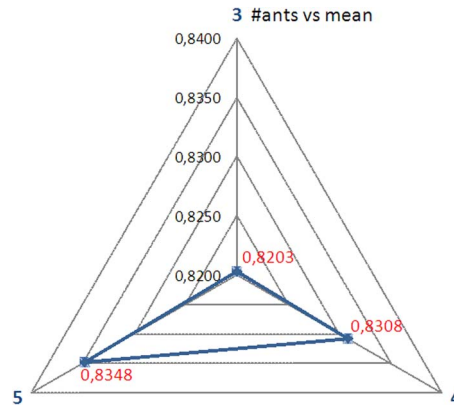


FIGURE 2. Experiment 1. Comparing the TCP-ANT mean values of 100 runs with three to five ants.

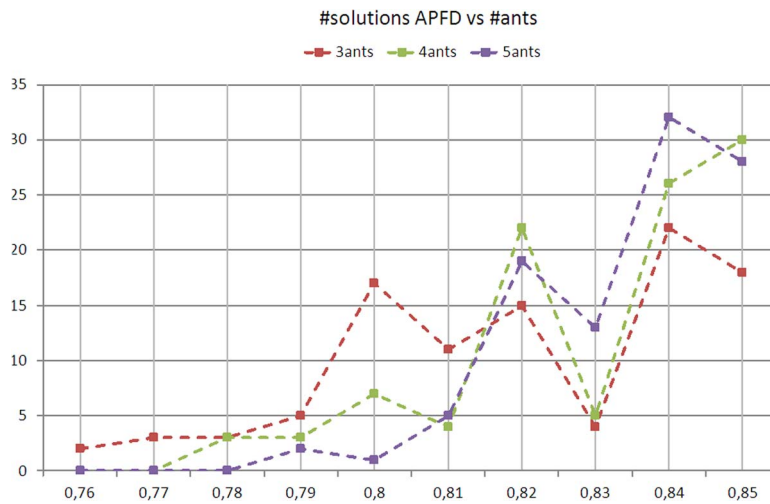


FIGURE 3. Experiment 1. The number of TCP-ANT solutions obtained with various APFD of 100 runs with different configurations, using three to five ants.

Figure 3 shows the number of solutions with various APFD values of all the runs for each experiment. The largest number of solutions with best APFD are obtained, in most of the cases, using the largest number of ants.

All the configurations found the solution with $APFD=0.85$, but with different solutions.

Several examples of the solution obtained for a specific number of ants follow: solution: $(T8, T4, T2, T1, T3, T5, T6, T7, T9, T10)$ with three ants, solution: $(T8, T4, T2, T6, T1, T3, T5, T9, T7, T10)$ with four ants and solution: $(T8, T4, T2, T1, T3, T5, T6, T7, T9, T10)$ with five ants.

4.2 Experiment 2. Theoretical configuration

This experiment is based on Experiment 1 by changing the number of test cases from 10 to 15 and using the same severity. Three different runs, with three to five ants, and 100 runs are performed. The best solutions in terms of APFD were recorded.

The obtained results are in agreement with the previous experiment, i.e. obtaining the best values with 5 ants: APFD = 0.8082 (for execution with 3 ants), APFD = 0.8144 (for execution with 4 ants) and APFD = 0.8157 (for execution with 5 ants). All executions obtained the best APFD = 0.8333.

4.3 Experiment 3. Theoretical configuration

Experiment 2 is a version of Experiment 1 version, with the same number of test cases but with the changed number of faults from 10 to 15, and the severity values modified from 1 to 8. Three different executions of 100 runs each, with three to five ants, are performed.

The best solutions in terms of APFD recorded for each run of the total 100 runs are in correlation with the Experiment 2 results: the best values with 5 ants: APFD = 0.7969 (for execution with 3 ants), APFD = 0.8039 (for execution with 4 ants) and APFD = 0.8065 (for execution with 5 ants). The overall executions obtained the best APFD = 0.8167.

4.4 Experiment 4. Project *grep*

The current experiment is based on the real-life *grep* project from SIR [9].

As we studied the objects available on SIR [9], we focused on the items with test specifications files based on TSL, making the test case and fault management for the TCP problem more fluently.

The *grep* objects met the prerequisites: includes the test information in the required format and provides a high number of test cases in a test suite.

For *grep* project there is a peak of 465 test cases per test suite and 75 faults (in the selected Test Suite just four were used). More details of the large dataset are available online <https://www.cs.ubbcluj.ro/~avesan/IGPL2020/>.

Data Preparation. In order to address the tests–faults relations and cost of each test case, several adjustments were made on original SIR tools. The achieved changes refer to the following:

- **traceability matrix** – we used the `.tsl` and `.frame` file (as described in [1]) to extract data required for test case traceability;
- **test case execution time** – the existing *mts* tool on SIR was adapted and used together with the original `mydate.c` and `time.pl` tools in order to provide proper information on time required to run a test case.

Regarding the *Severity* of the faults no data could be ‘discovered’. To overcome this situation, we assigned the faults severity according to the function criticality presented in [7], a measurement that assumes that faults exposed in a function with higher importance to the system are more severe than faults exposed in a less important function. Thus, are randomly assign value, an integer ranging from 1 to 8, as the severity of faults exposed.

Figure 4 illustrates the mean of all the 100 runs for each configurations with three to five ants. The latest configurations with five ants had the best results.

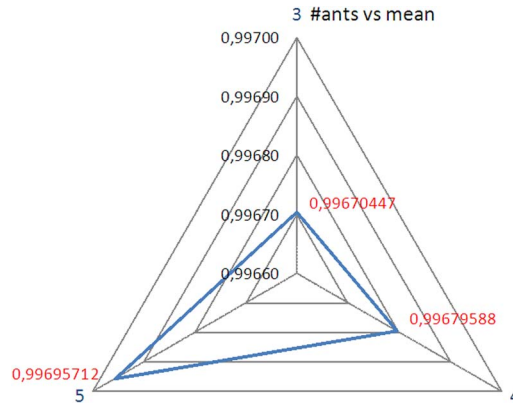


FIGURE 4. Experiment 4. Comparing the TCP-ANT mean values of 100 runs with three to five ants.

4.5 Discussion

The design of the experiments involves establishing the best value for parameters. The number of ants used in the algorithm was stated. Validation of the algorithm using different configurations and introducing a new benchmark was provided.

The experiments revealed that the best value for the number of ants is the largest one tested, five ants, i.e. such that the best results are obtained.

We applied the replication strategy as in [5] and performed various experiments, varying different elements of the test case prioritization problem. As the algorithm was successfully tested with different number of test cases and number of faults, and different severity values it could be considered robust.

5 Threats to Validity

The proposed approach for test case prioritization using ANT algorithm, as every experimental analysis, may suffer from some threats to validity and biases that can affect the results of our study. Furthermore, the issues influencing the results are analysed.

Threats to internal validity refer to the subjectivity introduced in setting the values for the severity of test cases. In order to minimize threats to internal validity, we conducted two different experiments with different random values.

Threats to external validity are related to the generalization of the obtained results. Just an experiment was provided in [8]; in the current work two more different experiments, including an open-source project, were tested with different severity values.

6 Conclusions and Further Work

Test case prioritization strategy was used in the current paper to solve the regression testing problem. The paper describes the *TCP-ANT*, an ant-based model used to find an optimal ordering solution. Severity of faults, a security feature, is included as a criterion when selecting the next test case; another one is the number of faults remained to be added. In order to reduce prioritization costs, a new criterion referring to the number of test cases to cover the faults is introduced.

Future work could improve the *TCP-ANT* by using beneficent metrics (related to the testing type) for large test cases suites as for gate assignment [26], railway TSP [27], sensor networks security [28], secure and green supply chain networks [29, 30], with tests–faults relations [4]. From the security perspective, the *severity* of an identified fault, each fault reviewing a ‘severity’ value was considered. Non-quality attributes [2, 3] will be considered in the prioritization. Other future related work could consider safety-critical, non-critical and unclassified faults.

References

- [1] T. J. Ostrand and M. J. Balcer. The category-partition method for specifying and generating functional tests. *Communications of the ACM*, **31**, 676–686, 1988.
- [2] A. Vescan and C. Serban. Multilevel component selection optimization toward an optimal architecture. *Soft Computing*, **21**, 4481–4495, 2017.
- [3] C. Serban and A. Vescan. *Predicting Reliability by Severity and Priority of Defects*, *ACM SIGSOFT Int. W. Soft. Qualities and Their Dependencies*, pp. 27–34, 2019.
- [4] A. Vescan, C. Serban, C. Chisalita-Cretu and L. Diosan. Requirement dependencies-based formal approach for test case prioritization in regression testing. *IEEE ICCP*, 181–188, 2017.
- [5] R. K. Yin. *Case Study Research: Design and Methods*, *Applied Social Research Methods*. Sage Publications, 2008.
- [6] G. Rothermel. et al. Test case prioritization: an empirical study. In *Software Maintenance*, pp. 179–188. IEEE ICSM 1999, 1999.
- [7] S. Elbaum, A. Malishevsky and G. Rothermel. Incorporating varying test costs and fault severities into test case prioritization. *ICSE*, **2001**, 329–338, 2001.
- [8] A. Vescan, C.-M. Pintea and P. C. Pop. Solving the test case prioritization problem with secure features using ant colony system. *CISIS*, **2020**, 67–76, 2019.
- [9] Software-artifact Infrastructure Repository, <http://sir.unl.edu/>, 2019.
- [10] M. Rava and W. M. Wan-Kadir. A review on prioritization techniques in regression testing. *International Journal of Software Engineering & Applications*, **10**, 221–232, 2016.
- [11] S. Yoo and M. Harman. Regression testing minimization, selection and prioritization: a survey. *Software Testing, Verification and Reliability*, **22**, 67–120, 2012.
- [12] A. P. Agrawal and A. Kaur. A comprehensive comparison of ant colony and hybrid particle swarm optimization algorithms through test case selection. *Data Engineering and Intelligent Computing*, 397–405, 2018.
- [13] S. F. Ahmad, D. K. Singh and P. Suman. *Prioritization for Regression Testing Using Ant Colony Optimization Based on Test Factors*, pp. 1353–1360. *Intell. Comm, Control and Devices*, 2018.
- [14] R. Battiti. *Reactive Search: Toward Self-Tuning Heuristics*, *Modern Heuristic Search Methods*, V. J. Rayward-Smith, ed., ch. 4. John Wiley & Sons Ltd, 1996.
- [15] M. Dorigo and L. M. Gambardella. Ant colony system: a cooperative learning approach to the traveling salesman problem. *IEEE Transactions on Evolutionary Computation*, **1**, 53–66, 1997.
- [16] S. G. Elbaum, A. G. Malishevsky and G. Rothermel. Prioritizing test cases for regression testing. *ISSTA*, **2000**, 102–112, 2000.
- [17] T. L. Graves. et al. An empirical study of regression test selection techniques. *ICSE*, **1998**, 188–197, 1998.
- [18] J. Hwang. et al. Selection of regression system tests for security policy evolution. *IEEE/ACM ASE*, 266–269, 2012.
- [19] M. I. Kayes. Test case prioritization for regression testing based on fault dependency. *International Conference on Electronics Computer Technology*, **5**, 48–52, 2011.

- [20] N. S. Kavitha. Test case prioritization for regression testing based on severity of fault. *International Journal of Computational Science and Engineering*, **2**, 1462–1466, 2010.
- [21] A. Khalilian, M. A. Azgomi and Y. Fazlalizadeh. An improved method for test case prioritization by incorporating historical test case data. *Science of Computer Programming*, **78**, 93–116, 2012.
- [22] Z. Li, M. Harman and R. M. Hierons. Search algorithms for regression test case prioritization. *IEEE Transactions on Software Engineering*, **33**, 225–237, 2007.
- [23] A. Malishevsky. et al. Cost-cognizant test case prioritization. *Nebraska Univ. Report TR-UNL-CSE-2006-004*, 2006.
- [24] OASISOASIS, eXtensible Access Control Markup Language, XACML, <http://www.oasis-open.org/committees/xacml>, 2005.
- [25] A. Perez-Uribe., *Ant Colony System algorithm in C/C++*, 2002.
- [26] C. M. Pintea, P. C. Pop, C. Chira and D. Dumitrescu. A hybrid ant-based system for gate assignment problem. In *Hybrid Artificial Intelligence Systems*, LNCS 5271, pp. 273–280. Springer, 2008.
- [27] P. C. Pop, C. M. Pintea and C. P. Sitar. An ant-based heuristic for the railway traveling salesman problem. In *EvoWorkshops 2007*, LNCS 4448, pp. 702–711. Springer, 2007.
- [28] C.-M. Pintea and P. C. Pop. Sensor networks security based on sensitive robots agents: a conceptual model. *AISC Springer*, **189**, 47–56, 2013.
- [29] C.-M. Pintea. et al. Towards secure & green two-stage supply chain networks. *Logic Journal of the IGPL*, 137–148, 2018.
- [30] C.-M. Pintea, G. C. Crişan and P. Pop. Towards secure transportation based on intelligent transport systems. *AISC Springer*, 469–477, 2018.
- [31] A. Holzinger. et al. A glass-box interactive machine learning approach for solving NP-hard problems with the human-in-the-loop. *Creative Mathematics and Informatics*, **28**, 121–134, 2019.
- [32] A. Holzinger. et al. Interactive machine learning: experimental evidence for the human in the algorithmic loop. *Applied Intelligence*, **49**, 2401–2414, 2019.
- [33] P. C. Pop. *Generalized Network Design Problems. Modeling and Optimization*. Publisher De Gruyter, 2012.
- [34] G. McGraw and B. Potter. Software Security Testing. *IEEE Security & Privacy*, **2**, 81–85, 2004.
- [35] G. Rothmel. et al. Test case prioritization: an empirical study. *ICSM*, 179–188, 1999.
- [36] P. Saraswat, A. Singhal and A. Bansal. A review of test case prioritization and optimization techniques. *AISC Springer*, 507–516, 2018.
- [37] Y. Singh, A. Kaur and B. Suri. Test case prioritization using ant colony optimization. *ACM SIGSOFT Software Engineering Notes*, **35**, 1–7, 2010.
- [38] X. Zhang. et al. Test case prioritization based on varying testing requirement priorities and test case costs. *QSIC*, **2007**, 15–24, 2007.

Received 1 March 2020

© 2022 Dov Gabbay. Copyright of Logic Journal of the IGPL is the property of Oxford University Press / USA and its content may not be copied or emailed to multiple sites or posted to a listserv without the copyright holder's express written permission. However, users may print, download, or email articles for individual use.